

CUDA Standard Algorithms » Parallel Find

Contents

- [Include the Header](#)
- [Find an Element in a Range](#)
- [Find the Minimum Element in a Range](#)
- [Find the Maximum Element in a Range](#)

Taskflow provides standalone template methods for finding elements in the given ranges using GPU.

Include the Header

You need to include the header file, `taskflow/cuda/algorithm/find.hpp`, for using the parallel-find algorithm.

```
#include <taskflow/cuda/algorithm/find.hpp>
```

Find an Element in a Range

tf::cuda_find_if finds the index of the first element in the range `[first, last)` that satisfies the given criteria. This is equivalent to the parallel execution of the following loop:

```
unsigned idx = 0;
for(; first != last; ++first, ++idx) {
    if (p(*first)) {
        return idx;
    }
}
return idx;
```

If no such an element is found, the size of the range is returned. The following code finds the index of the first element that is dividable by 17 over a range of one million elements.

```
const size_t N = 1000000;
auto vec = tf::cuda_malloc_shared<int>(N); // vector
auto idx = tf::cuda_malloc_shared<unsigned>(1); // index

// initializes the data
for(size_t i=0; i<N; vec[i++] = rand());

// create an execution policy
tf::cudaDefaultExecutionPolicy policy;

// finds the index of the first element that is a multiple of 17
tf::cuda_find_if(
    policy, vec, vec+N, idx, [] __device__ (auto v) { return v%17 == 0; }
);

// wait for the find operation to complete
stream.synchronize();

// verifies the result
if(*idx != N) {
    assert(vec[*idx] %17 == 0);
}

// deletes the memory
cudaFree(vec);
cudaFree(idx);
```

The find-if algorithm runs *asynchronously* through the stream specified in the execution policy. You need to synchronize the stream to obtain the correct result.

Find the Minimum Element in a Range

tf::cuda_min_element finds the index of the minimum element in the given range `[first, last)` using the given comparison function object. This is equivalent to a parallel execution of the following loop:

```
if(first == last) {
    return 0;
}
auto smallest = first;
for (++first; first != last; ++first) {
    if (op(*first, *smallest)) {
        smallest = first;
    }
}
return std::distance(first, smallest);
```

The following code finds the index of the minimum element in a range of one millions elements using GPU computing:

```
const size_t N = 1000000;
auto vec = tf::cuda_malloc_shared<int>(N); // vector
auto idx = tf::cuda_malloc_shared<unsigned>(1); // index

// initializes the data
for(size_t i=0; i<N; vec[i++] = rand());

// create an execution policy
tf::cudaStream stream;
tf::cudaDefaultExecutionPolicy policy(stream);

// queries the required buffer size to find the minimum element over N element
auto bytes = policy.min_element_bufsz<int>(N);
auto buffer = tf::cuda_malloc_device<std::byte>(bytes);

// finds the minimum element using the less comparator
tf::cuda_min_element(
    policy, vec, vec+N, idx, [] __device__ (auto a, auto b) { return a<b; }, buffer
```

```
);

// wait for the min-element operation completes
stream.synchronize();

// verifies the result
assert(vec[*idx] == *std::min_element(vec, vec+N, std::less<int>{}));

// deletes the memory
cudaFree(vec);
cudaFree(idx);
cudaFree(buffer);
```

Since the GPU min-element algorithm may require extra buffer to store the temporary results, you need to provide a buffer of size at least larger or equal to the value returned from **tf::cudaDefaultExecutionPolicy::min_element_bufsz**.

Attention

You must keep the buffer alive before the **tf::cuda_min_element** completes.

Find the Maximum Element in a Range

Similar to **tf::cuda_min_element**, **tf::cuda_max_element** finds the index of the maximum element in the given range `[first, last)` using the given comparison function object. This is equivalent to a parallel execution of the following loop:

```
if(first == last) {
    return 0;
}
auto largest = first;
for (++first; first != last; ++first) {
    if (op(*largest, *first)) {
        largest = first;
    }
}
return std::distance(first, largest);
```

The following code finds the index of the maximum element in a range of one millions elements using GPU computing:

```
const size_t N = 1000000;
auto vec = tf::cuda_malloc_shared<int>(N);      // vector
auto idx = tf::cuda_malloc_shared<unsigned>(1);  // index

// initializes the data
for(size_t i=0; i<N; vec[i++] = rand());

// create an execution policy
tf::cudaStream stream;
tf::cudaDefaultExecutionPolicy policy(stream);

// queries the required buffer size to find the maximum element over N element
auto bytes = policy.max_element_bufsz<int>(N);
auto buffer = tf::cuda_malloc_device<std::byte>(bytes);

// finds the maximum element using the less comparator
tf::cuda_max_element(
    policy, vec, vec+N, idx, [] __device__ (auto a, auto b) { return a<b; }, buffer
);

// wait for the max-element operation to complete
stream.synchronize();

// verifies the result
assert(vec[*idx] == *std::max_element(vec, vec+N, std::less<int>{}));

// deletes the memory
cudaFree(vec);
cudaFree(idx);
cudaFree(buffer);
```

Since the GPU max-element algorithm may require extra buffer to store the temporary results, you need to provide a buffer of size at least larger or equal to the value returned from **tf::cudaDefaultExecutionPolicy::max_element_bufsz**.

Attention

You must keep the buffer alive before **tf::cuda_max_element** completes.